

Sistema Multiagente para Generación y Validación de Consultas DAX

Arquitectura, decisiones de diseño e implementación

Lautaro Quispe Ballesteros

11 de julio de 2026

Agenda

- 1 Motivación y problema
- 2 Arquitectura general
- 3 Modelado de datos con Pydantic
- 4 MCP: Model Context Protocol
- 5 Validación de consultas DAX
- 6 Sistema multiagente
- 7 Orquestación con LangGraph
- 8 API y despliegue
- 9 Evaluación experimental
- 10 Resultados
- 11 Limitaciones y trabajo futuro
- 12 Cómo usar el sistema

El problema: generar DAX correcto es difícil

Dos cosas a la vez, no una

- Conocer la **sintaxis** de DAX (EVALUATE, CALCULATE, SUMMARIZECOLUMNS, FILTER...).
- Conocer la **estructura exacta** del modelo semántico contra el que se ejecuta: qué tablas existen, cómo se llaman sus columnas, qué relaciones las conectan, qué medidas ya están definidas.

Un LLM sin ese segundo ingrediente

Alucina: inventa nombres de tabla o columna plausibles pero inexistentes, o reescribe a mano una medida que ya existe en el modelo. Y lo hace sin ninguna señal de que se equivocó.

Por qué no alcanza con que el LLM se autoevalúe

- Un único LLM que genera y se autoevalúa en el mismo turno tiende a confirmar su propio trabajo (no hay un segundo juicio independiente).
- Comparar contra un DAX “esperado” con métricas léxicas (similaridad de texto) penaliza soluciones correctas pero escritas distinto.
- Hacen falta tres piezas: (1) una fuente de verdad del esquema real, (2) un juicio independiente del generador, (3) un mecanismo de corrección acotado.

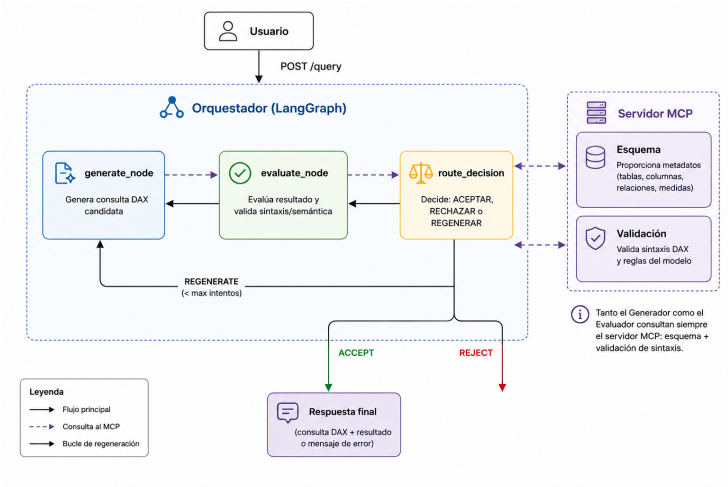
Pregunta técnica central

*¿En qué medida un agente evaluador basado en **LLM-as-a-Judge** con bucle de regeneración mejora la validez y coincidencia semántica de consultas DAX generadas, frente a generación directa sin evaluador?*

Propuesta

Dos agentes LLM (**generador** y **evaluador**) que consultan el modelo semántico real a través de un servidor **MCP**, conectados por un **bucle acotado de regeneración** (máx. 3 intentos).

Arquitectura del sistema



Flujo end-to-end: de la pregunta a la respuesta

- 1 El usuario hace POST /query con {"question": "..."} (api/routes.py).
- 2 El endpoint arma un AgentState inicial y llama build_graph().ainvoke(...) (orchestrator/graph.py).
- 3 generate_node pide el esquema al servidor MCP y genera DAX con el LLM.
- 4 evaluate_node pide el esquema completo + valida sintaxis, y el LLM evaluador juzga la consulta.
- 5 route_decision decide: ACCEPT/REJECT → fin; REGENERATE → vuelve a 3 con el feedback_for_generator.
- 6 finalize_node arma la respuesta final y la API la serializa como QueryResponse.

Organización del repositorio

<code>models/</code>	Modelos Pydantic de dominio (DAXQuery, EvaluationResult...)
<code>config/</code>	Settings (pydantic-settings) + logging estructurado
<code>mcp_server/</code>	Servidor MCP + cliente Power BI (mock)
<code>agents/</code>	BaseAgent, GeneratorAgent, EvaluatorAgent
<code>prompts/</code>	Construcción de prompts de cada agente
<code>orchestrator/</code>	Grafo de estados (LangGraph) que coordina el flujo
<code>api/</code>	API REST (FastAPI)
<code>tests/</code>	Fixtures (esquema + corpus) y pruebas (unit + integration)
<code>scripts/</code>	Línea base, sistema completo, comparación
<code>results/</code>	Métricas reales de las corridas
<code>notebook.ipynb</code>	Reproduce el experimento completo
<code>Dockerfile, docker-compose.yml</code>	

Rol de cada módulo

Módulo	Responsabilidad
models/	Contrato de datos único, validado, tipado
config/	Una sola fuente de configuración
mcp_server/	Única puerta de acceso al modelo semántico
agents/	Lógica de generación y de juicio
prompts/	Qué se le dice exactamente al LLM y por qué
orchestrator/	El bucle: cuándo parar, cuándo reintentar
api/	Punto de entrada HTTP
scripts/	Medir el sistema con números reales

Pydantic: contratos de datos entre módulos

El principio

Todo dato que cruza la frontera entre dos módulos es un modelo Pydantic: la forma se valida en el instante de entrada, no cuando el código intenta usarlo — y cada modelo expone la lógica que le corresponde.

Modelo	Archivo	Recorrido en el sistema
SchemaContext	schema_context.py	MCP client → agentes → prompts
DAXQuery	dax_query.py	Generador → Evaluador → API
EvaluationResult	evaluation_result.py	Evaluador → Orquestador → API
Settings	config/settings.py	Toda la aplicación (singleton)

Cada modelo incorpora su propia lógica de validación: `EvaluationResult` rechaza un score fuera de `[0,1]`; `Settings` falla al arrancar si `.env` contiene un tipo incorrecto; `DAXQuery` expone `starts_with_evaluate()` para no repetir esa comprobación en cada agente. Las diapositivas siguientes los recorren uno a uno.

SchemaContext: el contrato del modelo semántico

Archivo: models/schema_context.py

```
1 class SchemaContext(BaseModel):
2     tables: list[TableInfo]
3     relationships: list[RelationshipInfo]
4     measures: list[MeasureInfo]
5
6     def get_table(self, name: str) -> TableInfo | None:
7         for t in self.tables:
8             if t.name.lower() == name.lower():
9                 return t
10        return None
11
12    def to_prompt_text(self) -> str:
13        ... # texto plano legible por el LLM, no JSON crudo
14
```

`to_prompt_text()` arma el contexto que recibe el generador: texto ordenado en vez de JSON, para ahorrar tokens y mejorar la calidad de generación.

DAXQuery y EvaluationResult

Archivos: models/dax_query.py, models/evaluation_result.py

```
1 class Decision(str, Enum):      # hereda de str: serializa como "ACCEPT"
2     ACCEPT = "ACCEPT"
3     REJECT = "REJECT"
4     REGENERATE = "REGENERATE"
5
6 class EvaluationResult(BaseModel):
7     score: float = Field(..., ge=0.0, le=1.0)
8     decision: Decision
9     explanation: str
10    syntax_valid: bool = False
11    feedback_for_generator: str | None = None
12
```

ge=0.0, le=1.0 en score rechaza automáticamente un valor fuera de rango que el LLM evaluador pudiera devolver mal.

Settings: configuración centralizada y tipada

Archivo: config/settings.py (pydantic-settings)

```
1 class Settings(BaseSettings):
2     model_config = SettingsConfigDict(env_file=".env", extra="ignore")
3
4     gemini_api_key: str = ""
5     generator_model: str = "gemini-flash-latest"
6     max_iterations: int = 3
7     accept_threshold: float = 0.80
8     reject_throat: float = 0.50
9
10 settings = Settings()    # singleton: se crea una sola vez al importar
11
```

Si MAX_ITERATIONS en .env no fuera un número, el programa falla al arrancar, no a mitad de una corrida.

MCP: acceso al modelo semántico desde los agentes

El problema

Ambos agentes — generador y evaluador — necesitan el esquema, las relaciones, las medidas y validación DAX. Ese acceso tiene que ser limpio, tipado y reemplazable sin tocar los agentes (hoy mock, mañana Power BI XMLA real).

La solución: MCP

Protocolo estándar para invocar **herramientas externas tipadas** — funciones con nombre, parámetros y forma de retorno definidos — sin que el agente necesite saber cómo están implementadas por dentro.

- El agente recibe una lista fija de herramientas: `get_schema`, `validate_syntax`, ...
- Un **servidor** expone esas herramientas; un **cliente** las descubre y las invoca — el mismo patrón sin importar qué haya del otro lado.
- En este proyecto: los **agentes son el cliente**; `PowerBIClient` es lo que hay detrás del servidor.

La cadena de acceso: de agente a modelo semántico

```
GeneratorAgent --+
                  +--> BaseAgent --> ClientSession --> MCP Server (subprocess stdio)
EvaluatorAgent --+   connect_mcp()   call_tool()           |
                                                           +-----+-----+
                                                           | tools expuestas: |
                                                           |   get_schema      |
                                                           |   validate_syntax |
                                                           |   execute_dax ... |
                                                           +-----+-----+
                                                           |
                                                           PowerBIClient --> mock_schema.json
```

- Ambos agentes heredan de BaseAgent, que centraliza la conexión MCP y las llamadas a herramientas.
- El servidor MCP no es un servicio de red: es un **subproceso stdio** lanzado por el propio agente.
- PowerBIClient es el único punto de acceso al modelo semántico — hoy mock, mañana XMLA real.

Cómo arranca la conexión: connect_mcp()

Archivo: agents/base_agent.py

```
1 async def connect_mcp(self) -> None:
2     server_params = StdioServerParameters(
3         command="python",
4         args=["-m", "mcp_server.server"],
5     )
6     read, write = await stdio_client(server_params).__aenter__()
7     self._session = await ClientSession(read, write).__aenter__()
8     await self._session.initialize()    # handshake: cliente descubre las tools
9
```

- `python -m mcp_server.server` arranca como proceso hijo: el transporte es stdio estándar, no HTTP ni socket.
- `initialize()` realiza el handshake MCP — el cliente descubre qué herramientas expone el servidor antes de invocar cualquiera.
- Se llama **una sola vez por agente** al inicio de su ciclo de vida. Dos agentes = dos subprocesos independientes.

Las 6 herramientas MCP expuestas

Archivo: mcp_server/server.py

```
1 mcp = FastMCP("dax-mcp-server")
2
3 mcp.tool()(get_schema)           # SchemaContext completo
4 mcp.tool()(get_tables)           # solo tablas
5 mcp.tool()(get_relationships)    # solo relaciones
6 mcp.tool()(get_measures)         # solo medidas
7 mcp.tool()(validate_syntax)      # heurística de sintaxis DAX
8 mcp.tool()(execute_dax)          # ejecución simulada
9
10 if __name__ == "__main__":
11     mcp.run(transport="stdio")
12
```

Cada herramienta es una función libre en `mcp_server/tools/*.py` que delega en un cliente único — separar “herramienta MCP” de “cliente” permite cambiar de transporte sin tocar el cliente.

Cómo invoca un agente una herramienta

Archivo: agents/base_agent.py

```
1  async def call_tool(self, name: str, **kwargs: Any) -> Any:
2      if self._session is None:
3          await self.connect_mcp()
4
5      result = await self._session.call_tool(name, arguments=kwargs)
6      if result.isError:
7          raise RuntimeError(f"La herramienta MCP '{name}' fallo")
8
9      return json.loads(result.content[0].text)
10
```

- `result.content[0].text` es la representación JSON de lo que la herramienta devolvió — el formato uniforme en el que MCP transporta cualquier resultado.
- Si la sesión aún no existe, `connect_mcp()` la levanta en el primer uso; el resto de llamadas reutilizan la sesión ya abierta.

La arquitectura es exactamente la misma

Solo cambia el backend detrás del cliente. El protocolo MCP, las 6 herramientas, los agentes: nada de eso cambia el día que haya conexión real a Power BI.

- **Por qué mock:** Power BI Premium con soporte XMLA tiene costo de licenciamiento y requiere credenciales Azure AD activas — no disponibles en este contexto de desarrollo.
- El código ya deja el punto de extensión listo: `if not self.use_mock: raise NotImplementedError(...)` en cada método del cliente.
- Cambiar de mock a real implica reemplazar la implementación interna de ese cliente — ni el servidor MCP, ni los agentes, ni el orquestador cambian.

Cliente del modelo semántico: responsabilidades

Archivo: mcp_server/powerbi/client.py

```
1 class PowerBIClient:
2     def __init__(self, use_mock: bool = settings.use_mock) -> None:
3         self.use_mock = use_mock
4         self._schema_cache: SchemaContext | None = None
5
6     async def fetch_schema(self) -> SchemaContext:
7         if not self.use_mock:
8             raise NotImplementedError("Configura credenciales Azure en .env")
9         if self._schema_cache is None:
10             data = json.loads(Path(settings.mock_schema_path).read_text())
11             self._schema_cache = SchemaContext.model_validate(data)
12         return self._schema_cache
13
```

El esquema se cachea tras la primera lectura: todas las llamadas subsecuentes reusan el mismo objeto ya parseado.

La heurística de validate_dax

```
1 def _validate_heuristic(self, query, schema) -> ValidationResult:
2     text = query.strip()
3     if not text.upper().startswith("EVALUATE"):
4         return ValidationResult(valid=False, error="Debe iniciar con EVALUATE")
5     if text.count("(") != text.count(")"):
6         return ValidationResult(valid=False, error="Parentesis desbalanceados")
7     if text.count("[") != text.count("]"):
8         return ValidationResult(valid=False, error="Corchetes desbalanceados")
9     for table_name, _col in _extract_column_refs(text):
10         if schema.get_table(table_name) is None:
11             return ValidationResult(valid=False, error=f"Tabla '{table_name}' no existe")
12     return ValidationResult(valid=True)
13
```

Cuatro chequeos estructurales, nada más: palabra clave inicial, balanceo de paréntesis y corchetes, y existencia de las tablas referenciadas.

Por qué la validación no alcanza sola

La heurística no es un motor DAX

No compila la consulta. Verifica forma superficial (palabra clave, balanceo, existencia de tablas) — no verifica que las **columnas** referenciadas existan, ni que el tipo de retorno de la expresión sea el que EVALUATE necesita, ni la corrección semántica de la lógica.

- Una consulta puede pasar los cuatro chequeos y aun así no ejecutar en un motor DAX real.
- Ninguna heurística de sintaxis barata cubre, por diseño, errores semánticos (¿la consulta responde lo que se preguntó?) — para eso existe el evaluador LLM-as-a-Judge, no el validador.

Consecuencia de diseño

Si la validación posterior es necesariamente parcial, la prevención tiene que pasar por el **prompt del generador**: reglas de DAX explícitas y estrictas desde el principio, no solo un chequeo al final. Se retoma en la sección de prompts.

BaseAgent: lo que comparten los dos agentes

Archivo: agents/base_agent.py

```
1 class BaseAgent:
2     def __init__(self, model: str, temperature: float) -> None:
3         self.model = model
4         self.temperature = temperature
5         self._gemini = genai.Client(...) if settings.gemini_api_key else None
6         self._local = AsyncOpenAI(base_url=settings.local_base_url, ...)
7         self._session: ClientSession | None = None
8
9     async def connect_mcp(self) -> None: ...
10    async def call_tool(self, name: str, **kwargs) -> Any: ...
11    async def _chat(self, messages: list[dict], **kwargs) -> str: ...
12
```

GeneratorAgent y EvaluatorAgent heredan de acá: conexión MCP y chat con el LLM son responsabilidades comunes, no se duplican.

Resiliencia: proveedor cloud con respaldo local

```
1 async def _chat(self, messages, **kwargs) -> str:
2     if self._gemini is not None:
3         try:
4             return await self._chat_gemini(messages, **kwargs)
5         except Exception as exc:
6             log.warning("gemini_chat_failed_fallback_local", error=str(exc))
7     else:
8         log.info("gemini_api_key_no_configurada_usando_local")
9     return await self._chat_local(messages, **kwargs)
10
```

- Sin GEMINI_API_KEY configurada: va directo al modelo local, sin gastar reintentos en un camino que sabe que va a fallar.
- Con Gemini disponible pero fallando: 3 reintentos con backoff exponencial (tenacity) antes de degradar al modelo local vía Ollama.
- El sistema no depende de forma dura de un único proveedor cloud para poder responder.

GeneratorAgent: reunir contexto y generar

Archivo: agents/generator_agent.py

```
1 class GeneratorAgent(BaseAgent):
2     async def run(self, question: str, feedback: str | None = None) -> DAXQuery:
3         schema = await self._gather_context()
4         messages = build_generator_prompt(question, schema, feedback)
5         raw_response = await self._chat(messages)
6         query_text = self._extract_dax(raw_response)
7         return DAXQuery(query_text=query_text, natural_language=question,
8                         iteration=0 if feedback is None else 1)
9
10    async def _gather_context(self) -> SchemaContext:
11        tables = await self.call_tool("get_tables")
12        relationships = await self.call_tool("get_relationships")
13        measures = await self.call_tool("get_measures")
14        return SchemaContext.model_validate({...})
15
```

_gather_context pide 3 herramientas por separado, no get_schema de una vez: deja abierta la posibilidad de que, contra Power BI real, cada pieza tenga costo o latencia distintos.

Construcción del prompt del generador

Archivo: prompts/generator_prompts.py

```
1 GENERATOR_SYSTEM = """Eres un experto en DAX para Power BI.  
2 - Usa UNICAMENTE las tablas, columnas y medidas del esquema. Nunca inventes.  
3 - La consulta debe empezar siempre con EVALUATE.  
4 - Prioriza reutilizar medidas existentes en vez de recalcular con SUM/CALCULATE.  
5 - Devuelve UNICAMENTE el bloque de código DAX, sin explicaciones.  
6 - Si hay "CORRECCIONES REQUERIDAS", corrígelas exactamente."""  
7
```

Estas reglas son explícitas y estrictas **porque la validación posterior es parcial** (sección anterior): es más barato prevenir un error de esquema en el prompt que confiar en detectarlo después.

EvaluatorAgent: el patrón LLM-as-a-Judge

Archivo: agents/evaluator_agent.py

```
1 class EvaluatorAgent(BaseAgent):
2     async def run(self, question, dax_query) -> EvaluationResult:
3         schema = SchemaContext.model_validate(await self.call_tool("get_schema"))
4         syntax_result = await self.call_tool("validate_syntax",
5                                             dax_query=dax_query.query_text)
6         messages = build_evaluator_prompt(question, dax_query, schema, syntax_result)
7         raw = await self._chat(messages, temperature=0.0,
8                                response_format={"type": "json_object"})
9         try:
10             return EvaluationResult.model_validate(json.loads(raw))
11         except Exception:
12             return EvaluationResult(score=0.0, decision=Decision.REJECT,
13                                     explanation="JSON invalido del evaluador", ...)
```

temperature=0.0 explícito en la llamada: el evaluador debe ser determinista pase lo que pase con la configuración general. El try/except garantiza que una respuesta mal formada del LLM degrada a un veredicto válido en vez de propagar una excepción.

Prompt del evaluador: criterios y umbrales explícitos

Archivo: prompts/evaluator_prompts.py

- Criterios ponderados: coherencia semántica, conformidad de esquema, corrección sintáctica, eficiencia, completitud.
- Los umbrales de decisión se interpolan directo desde settings en el system prompt:

```
1 score >= {settings.accept_threshold}          -> ACCEPT
2 {settings.reject_threshold} <= score < accept_threshold -> REGENERATE
3 score < {settings.reject_threshold}           -> REJECT
4
```

El LLM evaluador y el orquestador miran la misma fuente de verdad — si se ajustan los umbrales, no hace falta tocar el texto del prompt a mano.

Sin LangGraph: orquestación imperativa

Sin LangGraph, la **lógica de negocio** (llamar a los agentes) y la **lógica de control** (decidir si parar o reintentar) coexisten en el mismo bloque:

```
1 feedback = None
2 iteration = 0
3
4 while iteration < max_iterations:
5
6     dax      = await generator.run(question, feedback)
7     evaluation = await evaluator.run(question, dax)
8
9     if evaluation.decision in (ACCEPT, REJECT):
10         break
11
12     feedback = evaluation.feedback_for_generator
13     iteration += 1
14
```

El programador implementa el ciclo, las condiciones de salida y el traspaso de contexto entre iteraciones. Todo mezclado en el mismo lugar.

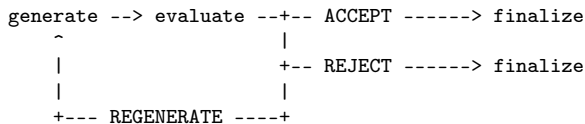
Con LangGraph: separar negocio de control

Lógica de negocio — qué hace cada nodo:

- `generate_node`: genera la consulta DAX.
- `evaluate_node`: evalúa la consulta.
- `finalize_node`: construye la respuesta final.

Lógica de control — la administra LangGraph:

- Qué nodo ejecutar a continuación.
- El estado compartido (`AgentState`).
- El ciclo de regeneración.



Ya no existe un `while`: el grafo determina la siguiente transición. El desarrollador declara el flujo; LangGraph lo ejecuta.

AgentState: el estado compartido entre nodos

Cada nodo recibe el estado actual y devuelve **solo los cambios**. LangGraph fusiona ese delta automáticamente.

```
1  # Estado inicial
2  state = { "question": "Ventas por categoria", "dax_query": None,
3            "evaluation": None, "iteration": 0, "max_iterations": 3 }
4
5  # Tras generate_node (devuelve solo el delta)
6  delta = { "dax_query": DAXQuery(...), "iteration": 1 }
7
8  # Tras evaluate_node (devuelve solo el delta)
9  delta = { "evaluation": EvaluationResult(decision=REGENERATE, score=0.65, ...) }
10
```

Si la decisión es REGENERATE, el estado vuelve a generate_node, que extrae:

```
1  feedback = state["evaluation"].feedback_for_generator # observaciones del evaluador
2  dax      = await generator.run(question, feedback)    # nueva consulta corregida
3
```

build_graph(): declarar el flujo de ejecución

```
1 graph.add_node("generate", generate_node)
2 graph.add_node("evaluate", evaluate_node)
3 graph.add_node("finalize", finalize_node)
4
5 graph.add_edge("generate", "evaluate")    # siempre evaluar tras generar
6
7 graph.add_conditional_edges(
8     "evaluate",
9     route_decision,
10    {
11        "ACCEPT":      "finalize",
12        "REJECT":      "finalize",
13        "REGENERATE": "generate",
14    },
15 )
16
```

route_decision() incluye la **garantía de terminación**: si la decisión es REGENERATE y `iteration >= max_iterations`, devuelve "REJECT". El desarrollador declara nodos y reglas de transición; no escribe un while. Esa es la ventaja principal de LangGraph en este proyecto.

La API: qué expone y cómo se usa

Endpoint	Método	Para qué
/health	GET	Confirmar que el servicio está arriba
/schema	GET	Ver el modelo semántico servido
/query	POST	Pregunta en lenguaje natural → DAX validado

```
1 POST /query
2 {"question": "Total de ventas por categoria de producto"}
3
4 200 OK
5 {"dax": "EVALUATE ...", "score": 0.9, "decision": "ACCEPT",
6  "iterations": 1, "explanation": "...", "syntax_valid": true}
7
```

Un único endpoint concentra todo el pipeline: quien consume la API no necesita saber que por detrás hay un bucle de generación y evaluación.

La API: cómo está implementada

- **FastAPI** (`api/routes.py` + `api/main.py`) como capa HTTP delgada sobre el grafo del orquestador — el endpoint arma el estado inicial, invoca el grafo, traduce el resultado.
- El ciclo de vida de la aplicación (*lifespan*) configura el logging al arrancar y cierra las sesiones MCP de los agentes al apagar — una sola vez por proceso, no en cada request.
- `/schema` consulta el cliente del modelo semántico directamente, sin pasar por MCP: API y servidor MCP corren en el mismo proceso, y son datos de solo lectura ya en memoria.
- CORS abierto: es un servicio de demostración/evaluación, no expone datos sensibles de producción.

Dockerfile

```
1 FROM python:3.11-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 EXPOSE 8000
7 CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
8
```

Un solo contenedor: el servidor MCP no necesita uno propio, se levanta como subproceso interno igual que en desarrollo local. `.dockerignore` excluye `.env` de la imagen.

```
1 environment:
2   LOCAL_BASE_URL: http://host.docker.internal:11434/v1
3
```

- Dentro de un contenedor, `localhost` apunta al contenedor mismo, no a la máquina anfitriona.
- `host.docker.internal` es el nombre que Docker Desktop resuelve hacia el host — necesario para que el respaldo local (Ollama, corriendo en la máquina real) siga siendo alcanzable desde adentro del contenedor.

El corpus de evaluación: 20 preguntas, 5 categorías

Archivo: tests/fixtures/corpus.json

Categoría	Preguntas
agregacion_simple	4
agregacion_con_filtro	4
inteligencia_temporal	4
cruce_tablas	4
reutilizacion_medidas	4

Distribución equilibrada a propósito: un corpus desbalanceado hacia lo simple ocultaría debilidades concentradas en una categoría difícil.

scripts/run_baseline.py: la línea base

```
1 agent = GeneratorAgent(model=settings.generator_model, ...)
2 for item in corpus:
3     dax_query = await agent.run(item["question"])
4     validation = await agent.call_tool("validate_syntax",
5                                       dax_query=dax_query.query_text)
6     ...
7     # guarda results/baseline.json
8
```

Corre solo el generador: sin evaluador, sin bucle, una pasada por pregunta. Es el punto de comparación necesario para que cualquier métrica del sistema completo signifique algo.

scripts/run_full_system.py: el sistema completo

```
1 final_state = await asyncio.wait_for(graph.ainvoke(initial_state), timeout=480)
2 ...
3 error_type = _classify_error(final_result) # sintaxis_invalida /
4                                             # fallo_parseo_evaluador /
5                                             # rechazado_por_evaluador
6
```

Corre el grafo completo (generador + evaluador + bucle) por cada pregunta. El timeout y el manejo de excepción por pregunta acotan el costo de una corrida sobre 20 preguntas frente a un LLM externo.

scripts/compare.py: línea base vs. sistema completo

```
1 rows = [  
2     ("% sintacticamente validas", baseline["pct_syntax_valid"], system["pct_syntax_valid"]),  
3     ("% aceptadas (ACCEPT)", baseline["pct_syntax_valid"], system["pct_accepted"]),  
4     ("Iteraciones promedio", 1.0, system["avg_iterations"]),  
5 ]  
6
```

La línea base no tiene evaluador: no existe un ACCEPT/REJECT real ni iteraciones reales. Se usan dos convenciones explícitas, documentadas en el propio `comparison.md`, para poder comparar ambas condiciones en la misma tabla.

Métrica	Línea base	Sistema completo
% sintácticamente válidas	100 %	95 %
% aceptadas (ACCEPT)	100 %*	75 %
Score promedio del evaluador	N/A	0.79
Iteraciones promedio	1.0*	1.7

*Convención: la línea base no tiene evaluador ni bucle propio.

Caída del 100 % al 75 %

La línea base mide “válida” con una heurística de sintaxis superficial. El sistema completo mide “aceptada” con un juicio semántico real, más estricto por diseño — el número más bajo refleja un criterio más exigente, no una regresión de calidad.

Resultados por categoría

Categoría	Válidas (sist.)	Aceptadas (sist.)
agregacion_simple	4/4	4/4
agregacion_con_filtro	4/4	4/4
inteligencia_temporal	4/4	4/4
cruce_tablas	3/4	0/4
reutilizacion_medidas	4/4	3/4

`cruce_tablas` (combinar 3+ tablas por relaciones) concentra toda la dificultad: 0 de 4 aceptadas en esta corrida. El resto de las categorías se resuelve casi sin fricción.

Tipo de error	Cantidad
rechazado_por_evaluador	4
sintaxis_invalida	1

- La mayoría de los rechazos son semánticos, no de sintaxis — la señal esperada si el evaluador está cumpliendo el rol para el que se diseñó.
- Clasificación calculada en `scripts/run_full_system.py::_classify_error`, a partir de `syntax_valid` y el contenido de `explanation`.

- **sin conexión a Power BI Premium/XMLA**; validación y ejecución son heurísticas, no un motor DAX real.
- **Validador parcial**: no modela tipos tabla/escalar de EVALUATE, ni verifica existencia de columnas — solo tablas.
- **No determinismo del generador**: `temperature=0.2` produce resultados distintos entre corridas del mismo corpus.
- **Nombres de modelo cloud no siempre estables**: se mitiga usando un alias mantenido por el proveedor (`gemini-flash-latest`) en vez de fijar una versión puntual.
- **Respaldo local**: más lento (CPU) y de menor calidad que un modelo cloud; ocasionalmente devuelve campos de explicación vacíos.
- **Cuotas del proveedor cloud**: los planes gratuitos tienen límites diarios de requests por modelo.

- Validación sintáctica contra un parser DAX real (no heurístico).
- Modelar tipos tabla/escalar en el validador.
- Registrar, por cada llamada, qué backend respondió realmente (hoy la etiqueta de backend es a nivel de corrida, no por llamada individual).
- Conexión real a Power BI Premium vía XMLA, reemplazando el modo mock sin tocar agentes ni orquestador.
- Soporte multi-turno en la construcción de mensajes hacia el proveedor cloud (hoy asume un único mensaje de sistema + uno de usuario).
- Evaluar la salida de la línea base con el mismo juez LLM, para aislar el efecto puro del evaluador del efecto del bucle de regeneración.

Respondiendo la pregunta central

El evaluador LLM-as-a-Judge con bucle de regeneración no aumenta una tasa ingenua de “validez” — la redefine con un criterio más estricto y semánticamente informado, y usa el bucle para corregir activamente lo que ese criterio detecta.

- La arquitectura (MCP + Pydantic + LangGraph) permite reemplazar piezas (mock → real, proveedor cloud → local) sin reescribir agentes.
- El punto más débil identificado — cruzar múltiples tablas — es información concreta y accionable para la siguiente iteración del proyecto.

Gracias.